



FEDERICO II UNIVERSITY OF NAPLES

DEGREE **COURSE IN COMPUTER ENGINEERING**

SOFTWARE ENGINEERING COURSE

PROF. AR FASOLINO - AY 2024 - 25

WE POET



By the candidates:

Capponcelli Nicole Yuki N46006700

De Gregorio Daniele N46006631

Cozzarelli Riccardo N46003999

Summary

1. INFORMAL SPECIFICATIONS	4
2. ANALYSIS AND SPECIFICATION OF REQUIREMENTS	6
2.1. ANALYSIS OF NOUNS AND VERBS	6
2.2. VALIDATION OF REQUIREMENTS	8
2.3. GLOSSARY OF TERMS	9
2.4. CLASSIFICATION OF REQUIREMENTS	10
2.4.1. FUNCTIONAL REQUIREMENTS	10
2.4.2. DATA REQUIREMENTS	11
2.4.3. CONSTRAINTS AND OTHER REQUIREMENTS	11
2.5. USE CASE MODELING	12
2.5.1. ACTORS	12
2.5.2. USE CASES	12
2.5.3. TABLE OF USE CASES AND REQUIREMENTS	13
2.5.4. USE CASE DIAGRAM	14
2.5.5. SCENARIOS	15
2.6. CLASS DIAGRAM	18
2.7. SEQUENCE DIAGRAMS	19
2.7.2. VIEW STATISTICS	20
2.7.3. GENERATE REPORT	21
2.8. REFINED CLASS DIAGRAM	22
3. FUNCTIONAL TEST PLAN	23
3.1. PUBLIC POETRY	23
3.2. VIEW STATISTICS	26
3.3. GENERATE REPORT	28
4. DESIGN	31
4.1. CLASS DIAGRAM	31
4.1.1. TRANSLATION OF CLASSES AND ASSOCIATIONS	31
4.1.2. BCED PATTERN	33
4.2. SEQUENCE DIAGRAMS	36
4.2.1. PUBLIC POETRY	36
4.2.2. VIEW STATISTICS	37
4.2.3. GENERATE REPORT (2 SCREEN)	38
5. IMPLEMENTATION	40
5.1. PACKAGE DATABASE	40
5.2. PACKAGE ENTITY	41
5.3. PACKAGE CONTROL	41
5.4. PACKAGE BOUNDARY	42

5.5. PACKAGE DTO-----	42
5.6. DEPLOYMENT DIAGRAM -----	43
6. TESTING -----	44
6.1. STRUCTURAL TEST -----	44
6.1.1. CYCLOMATIC COMPLEXITY -----	44
6.2. JUNIT -----	46
6.3. FUNCTIONAL TEST -----	48

1. Informal specifications

Bring back the assigned track

We want to develop a software system for managing a social platform dedicated to writing and sharing short poems.

The platform will allow users to publish their own short poems, organize thematic collections, and interact with others' content through likes and comments.

The system allows user registration via email, first name, last name, and password.

Each user has an editable personal profile, which includes their personal data, a short biography and possibly a profile picture.

(identified by an image file).

Once authenticated, the user can publish, through a dedicated graphical interface, new short poems (maximum 500 characters), assigning each a title, one or more thematic tags (e.g., "love," "night," "solitude") and choosing whether to include it in an existing collection or in a new collection to be created on the spot.

The user can also choose to view the list of his or her collections and the list of all published poems.

Each collection is identified by a title and a description, and is composed of a set of poems by the same author.

Poems can be made public or private. Public poems are visible in users' feeds.

Once authenticated, each author can choose to display a list of new poems published by other authors in their main feed.

The personal feed displays a maximum of five poems, corresponding to the latest publications made by other authors, sorted in descending chronological order.

Users can interact with other people's poems by expressing their appreciation (a "heart") and possibly leaving a text comment.

Each comment is identified by author, text, and date.

When a poem is viewed, the total number of hearts it has received and a list of the last three comments it has received are also displayed.

Each user has access to a "Statistics" section that shows the total number of hearts received by their poems, the number of comments received on their poems, and the most liked poem (i.e. the one that received the most hearts).

Platform administrators (upon registration and authentication) can generate reports containing aggregate data such as the number of poems published in a

certain time interval, the most active authors (who have published the most total number of poems) and possibly the most used tags and the poems with the most interactions.

The platform must be accessible from both desktop and mobile devices, provide real-time notifications (for new comments or hearts received), and guarantee the protection of personal data, the security of stored information, and a robust authentication system.

2. Requirements analysis and specification

2.1. ANALYSIS OF NOUNS AND VERBS

We want to develop a software system for managing a social platform dedicated to writing and sharing short poems.

The platform will allow users to publish their own short poems, organize thematic collections, and interact with others' content through likes and comments.

The system allows user registration via email, first name, last name, and password.

Each user has a personal, editable profile, which includes their data personal details, a short biography and possibly a profile picture (identified by an image file).

Once authenticated, the user can publish, through a dedicated graphical interface, new short poems (maximum 500 characters), assigning each a title, one or more thematic tags (e.g., "love," "night," "solitude") and choosing whether to include it in an existing collection or in a new collection to be created on the spot.

The user can also choose to view the list of his or her collections and the list of all published poems.

Each collection is identified by a title and a description, and is composed of a set of poems by the same author.

Poems can be made public or private. Public poems are visible in users' feeds.

Once authenticated, each author can choose to display a list of new poems published by other authors in their main feed.

The personal feed displays a maximum of five poems, corresponding to the latest publications made by other authors, sorted in descending chronological order.

Users can interact with other people's poems by expressing their appreciation (a "heart") and possibly leaving a text comment

Each comment is identified by author, text, and date.

When a poem is viewed, the total number of hearts it has received and a list of the last three comments it has received are also displayed.

Each user has access to a "Statistics" section that shows the total number of hearts received by their poems, the number of comments received on their poems, and the most liked poem (i.e. the one that received the most hearts).

Platform administrators (upon registration and authentication) can generate reports containing aggregate data such as the number of poems published in a

a given time interval, the most active authors (who have published the greatest number of total poems) and possibly the most used tags and the poems with the most interactions.

The platform must be accessible from both desktop and mobile devices, provide real-time notifications (for new comments or hearts received), and guarantee the protection of personal data, the security of stored information, and a robust authentication system.

Legend:

Classes

Attributes

Actors

Functionality

Actor-Class

2.2. VALIDATION OF REQUIREMENTS

The assigned outline will be reported, revised following a simulated interaction with stakeholders, which will be useful for requirements analysis purposes in order to reveal errors, missing information, or conflicting or unachievable requirements.

1. The system must allow the User to register
2. For each User we want to store email, name, surname and password
3. The system must allow the User to access his/her profile
by authenticating with email and password
4. For each personal profile you want to store a biography, personal data and
a profile picture
5. The system must allow the User to modify the profile
6. The system must allow the User to publish Poems
7. For each poem you want to save the title, tags and Collection it belongs to.
8. The system must allow the user to create one or more collections
9. Each poem must be part of a collection
10. Each poem must be a maximum of 500 characters
11. The system must allow the User to view the list of his/her own
Collections
12. The system must allow the User to view the list of Poems
published
13. For each Collection you want to memorize the title, description and set of Poems,
by the same author, which are part of it
14. Poems must be able to be public or private
15. If a Poem is public, it can be viewed in other Users' Feeds
16. The system must allow the User to view his own feed
17. A User's Feed must be able to display the 5 most recently published poems by other Users in descending
chronological order (the first one that appears at the top is the most recent of all)
18. The system must allow users to leave a heart if necessary.
to other people's poems
19. The system must allow Users to leave a message if necessary
Commentary on Other People's Poems
20. For each comment we want to store the author who left it, the text and the
date
21. If a Poem is displayed, the total number of hearts and a list of the 3 most recent comments related to
the Poem must also be visible.
22. The system must allow the User to access a "Statistics" section
23. The "Statistics" section must show the total number of hearts received by all Poems posted by that User,
the number of comments received by all Poems posted by that User, and the Poem with the most hearts
of all.

- 24. The system must allow administrators to register and authenticate
- 25. The system must allow administrators to generate reports
- 26. The Report can contain the number of Poems published in a certain period of time, the most active Users, the most used tags and the Poems with the most interactions
- 27. The system must be accessible from both desktop and mobile devices
- 28. The system must provide real-time notifications for hearts and comments received
- 29. The system must ensure the protection of personal data, the security of stored information, and a robust authentication system.

2.3. GLOSSARY OF TERMS

Term	Description	Synonyms
User	Generic person	-
Author	User registered on the platform who can publish, view, and interact with poems	-
Poetry	An essay of up to 500 characters	Publication
Collection	Collection of poems published by a user	-
Feed	List of the 5 most recent poems published by other authors	-
Comment	Set of characters written by a user for another User's Poem	-
Heart	Interaction to express appreciation on the displayed poem	Appreciation
Tags	A single word to possibly thematize your poem	-
Report	Platform activity report that can only be generated by administrators	-
Administrator	Specialized User	-

2.4. CLASSIFICATION OF REQUIREMENTS

2.4.1. FUNCTIONAL REQUIREMENTS

Necessary to describe the services and features that the system will have to offer.

ID	Requirement	sentence no. in validation
RF01	The system must allow the User to register	1
RF02	The system must allow the User to access his/her profile by authenticating with email and password	3
RF03	The system must allow the User to modify the profile	5
RF04	The system must allow the User to publish Poems	6
RF05	The system must allow the User to create one or more collections	8
RF06	The system must allow the User to view the list of his/her Collections	11
RF07	The system must allow the User to view the list of published poems	12
RF08	The system must allow the User to view his own feed 16	
RF09	The system must allow users to leave possibly a heart to other people's poems	18
RF10	The system must allow users to leave comments on other people's poems.	19
RF11	The system must allow the User to access a section "Statistics"	22
RF12	The system must allow administrators to register and authenticate	24
RF13	The system must allow administrators to be able to generate Report	25
RF14	The system must provide real-time notifications for hearts and comments received	28

2.4.2. DATA REQUIREMENTS

ID	Requirement	sentence
		number in valid
RD01	For each user, we want to store email, name, surname and password	2
RD02	For each personal profile you want to store a biography, personal data and a profile image	4
RD03	For each poem you want to save the title, tags and Collection of belonging	7
RD04	For each Collection you want to store the title, description and set of Poems, by the same author, that are part of it	13
RD05	For each comment you want to store the author who left it, the text and the date	20
RD06	The "Stats" section must show the total number of hearts received from all the Poems published by that User, the number of comments received from all the Poems published by that User, and the Poem with the most hearts of all	23
RD07	The Report may contain the number of Poems published in a certain time interval, the most active users, the most used tags and the Poems with more interactions	26

2.4.3. CONSTRAINTS AND OTHER REQUIREMENTS

ID	Requirement	sentence
		number in valid
V01	Each poem must be part of a collection	9
V02	Each poem must be a maximum of 500 characters	10
RNF01	Poems must be able to be public or private	14
RNF02	If a Poem is public, it can be viewed in other people's feeds Users	15
V03	The 5 most popular poems must be able to appear in a User's Feed recently posted by other users in descending chronological order	17
V04	If a Poem is displayed it must be visible also the total number of hearts and a list of the 3 most popular comments recent related to Poetry	21
RNF03	The system must be accessible from both desktop and mobile devices	27
RNF04	The system must provide real-time notifications for hearts and comments received	28
RNF05	The system must ensure the protection of personal data, the security of stored information and a robust authentication system.	29

2.5. USE CASE MODELING

2.5.1. ACTORS

The primary actors are those who provide the stimulus for the use case to be launched, and they are:

- or User
- or Author
- or Administrator

Secondary actors, on the other hand, are those that interact with the use case that has already been started, and they are:

- or Email Service

2.5.2. USE CASES

These are the units of useful work that the system must be able to execute, and they are:

- o UC1: Registration o UC2: Access
- or UC3: Edit Profile
- o UC4: Publish Poetry o UC5: Create Collection o UC6: View Published Poems
- or UC7: ViewFeed
- or UC8: LeaveHeart
- or UC9: Leave a Comment
- or UC10: View Statistics
- o UC11: GenerateReport o UC12: ViewCollection

Inclusion use cases:

- o UC13: Send Confirmation (UC1) o UC14: Select Collection (UC4) o UC15: Assign Tag (UC4) o UC16: Assign Visibility (UC4) o UC17: Insert Report Data (UC11) o UC18: Insert Collection Description (UC5)

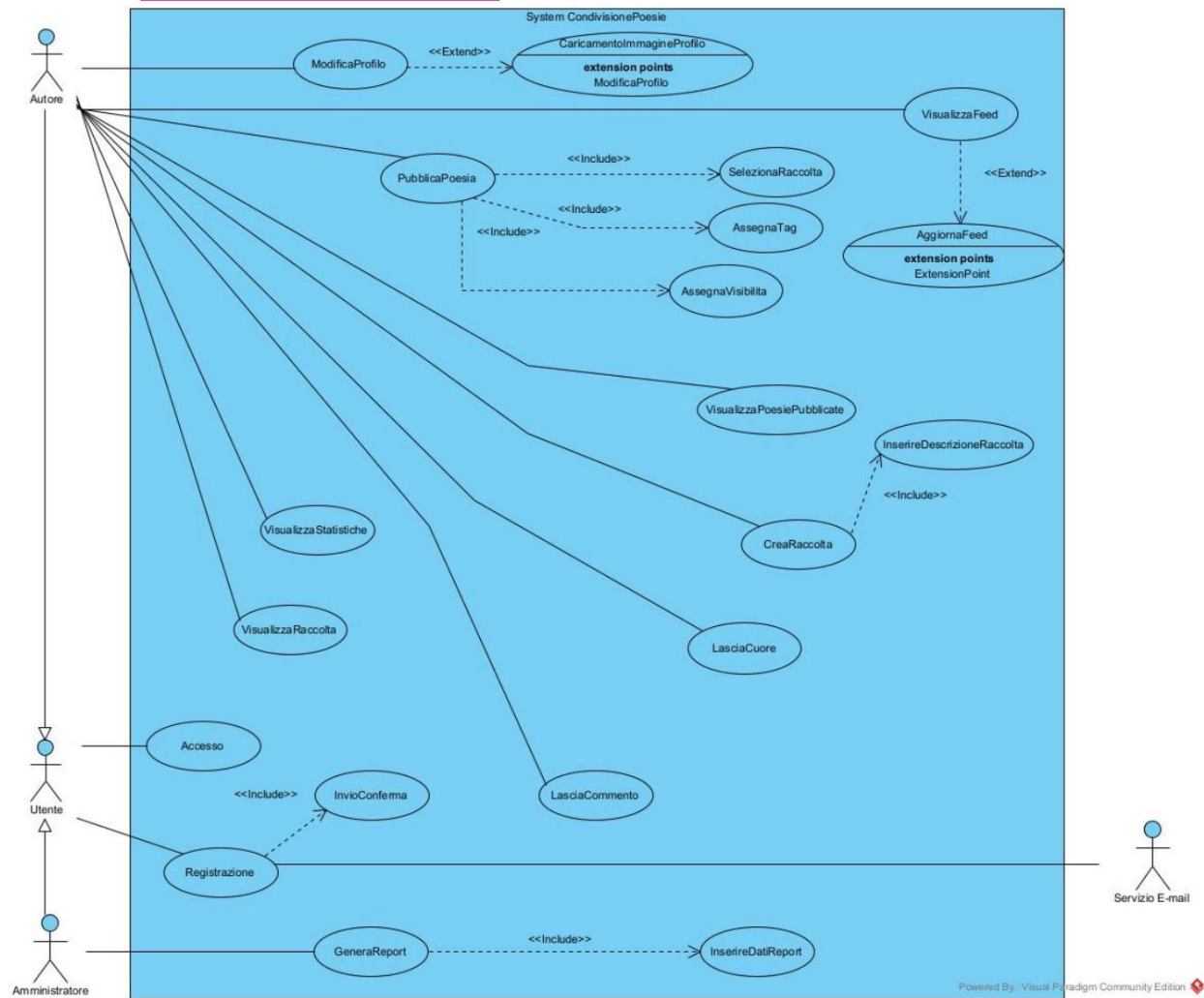
Extension use cases: (optional or exceptional events)

- o UC19: Load ProfileImage (UC3) o UC20: Update Feed (UC7)

2.5.3. TABLE OF USE CASES AND REQUIREMENTS

Use Case	Main Actors	Actors Secondary	Inclusion/ Extension	Related requirements
UC1	User	Email Service Includes	UC13	RF01, RF12
UC2	User	-	-	RF02
UC3	Author	-	Extends UC19	RF03
UC4	Author	-	Includes UC14, UC15, UC16	RF04
UC5	Author	-	Includes UC18 RF05	
UC6	Author	-	-	RF07
UC7	Author	-	Extends UC20	RF08
UC8	Author	-	-	RF09
UC9	Author	-	-	RF10
UC10	Author	-	-	RF11
UC11	Administrator	-	Includes UC17	RF13
UC12	Author	-	-	RF06
UC13	Author	Email Service Included in	UC1	RF01, RF12
UC14	Author	-	Included in UC4 RF04	
UC15	Author	-	Included in UC4 RF04	
UC16	Author		Included in UC4 RF04	
UC17	Author		Included in UC11 RF13	
UC18	Author		Included in UC5 RF05	
UC19	Author		Extended from UC03 RF03	
UC20	Author		Extended from UC07 RF08	

2.5.4. USE CASE DIAGRAM



2.5.5. SCENARIOS

2.5.5.1. PUBLICPOETRY USE CASE (UC4)

Use Case	UC4: Publish Poetry
Actor	User
Primary Actor	-
Secondary Actor	
Description	The user chooses to publish one of his short poems on the platform
Pre-Conditions	User must be logged in
Main sequence of events	1. Insert the poem to be published 2. Give the poem a title 3. Assign one or more thematic tags <<include>>AssignTag 4. Include it in a collection 4.1. if the desired collection already exists <<include>>SelectCollection 5. Assign visibility to the poem <<include>>AssignVisibility
Post-Conditions	The system publishes the poem and saves it in the database
Related use cases	AssignTag, SelectCollection, AssignVisibility
Sequence of alternative events	At point 1. , If the poem exceeds 500 characters, the system generates a error message. At point 4.1. , if the collection in which you want to include the poem is not present, you can create one on the spot with a title and possibly a description. <<include>>CreateCollection, <<include>>InsertCollectionDescription

2.5.5.2. VIEWSTATISTICS USE CASE (UC10)

Case	UC10: View Statistics
------	-----------------------

<i>of use</i>	
<i>Actor</i>	Author
<i>Primary</i>	
<i>Actor</i>	
<i>Second</i>	
<i>river</i>	
<i>Description</i>	The Author views the statistics relating to his poems: total hearts received, total number of comments received and most liked poem
<i>no</i>	
<i>Pre-</i>	Author must be logged in
<i>Condition</i>	
<i>no</i>	
<i>Main</i>	1. The use case starts when the user accesses the "Statistics" section
<i>sequence</i>	2. The system finds the poems published by the author
<i>of events</i>	3. The system calculates the total number of hearts received
	4. The system calculates the total number of comments received
	5. The System finds the most popular poem
	6. The system presents the calculated statistics to the user
<i>Post-</i>	Statistics are displayed, if available; otherwise, an informational message is displayed
<i>Conditions</i>	indicating that no data is available.
<i>Related use</i>	-
<i>cases</i>	
<i>Sequence of</i>	At point 2. if the Author has not yet published any poems, an information message will be
<i>alternative</i>	displayed ("No poems ")
<i>events</i>	At point 5. In case of a tie between poems, the system resolves the situation by showing the
	most recent one.
	At point 6. if the author has not received hearts but has published poems, an informational
	message will be displayed ("Your Poems Have Not Yet Received Hearts")

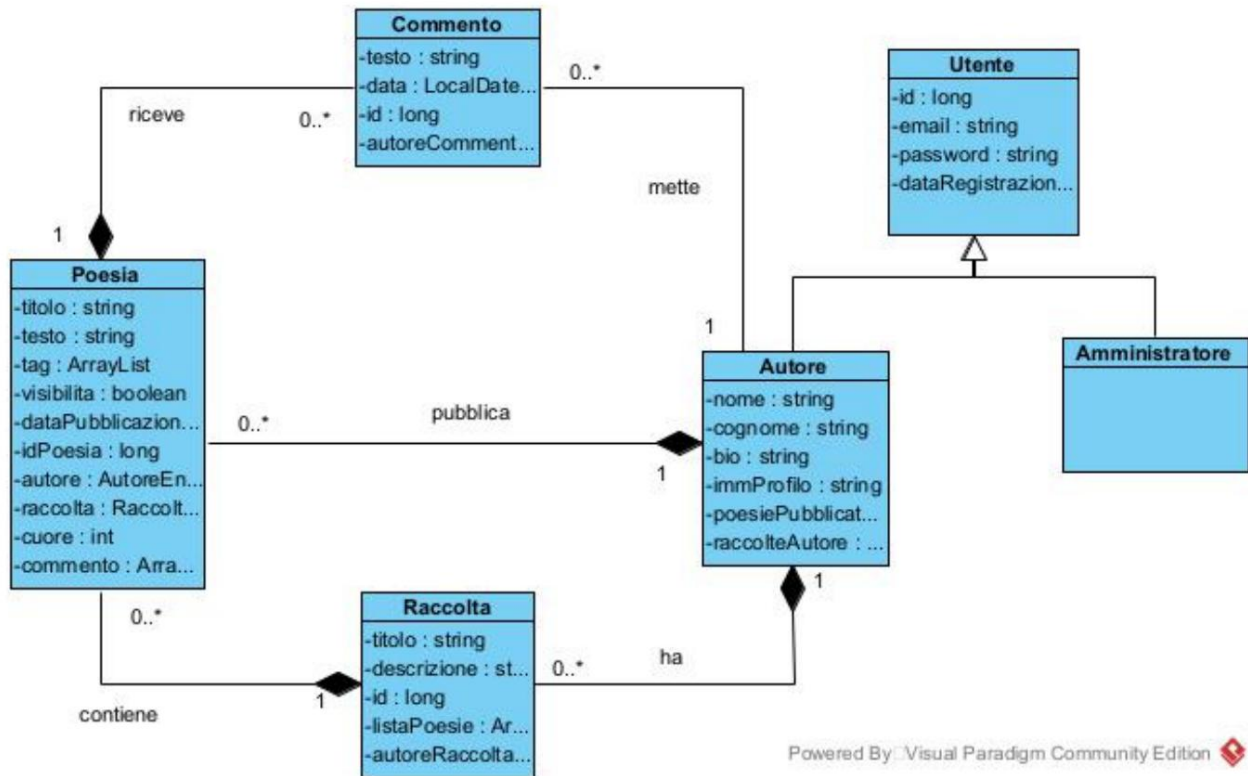
2.5.5.3. USE CASE GENERATEREPORT(UC11)

Case
In Use

UC11: GenerateReport

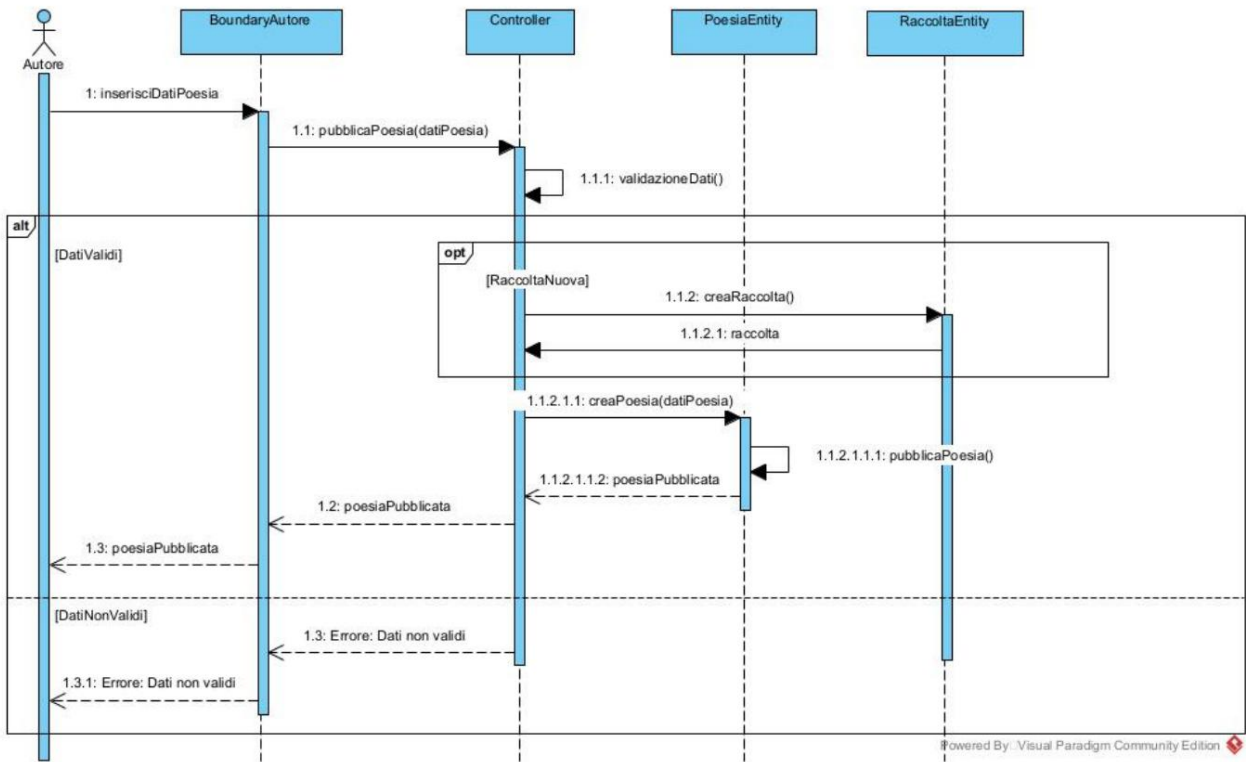
Actor Primary	Administrator
Actor Secondary	
Description	Administrators, after registration and authentication, can Generate reports containing aggregated data such as the number of poems published in a given time period, the most active authors, the most used tags, and the poems with the most interactions.
Pre-Conditions	Administrator must be logged in
Sequence of main events	1. The use case starts when the administrator accesses the section generate report 2. The administrator chooses which report he wants to generate 3. if the administrator chooses ReportPoesieIntervallo 3.1 The administrator enters the time interval 3.2 <<include>> InsertReportData 3.3 The system calculates the number of poems in the time interval 4. if the administrator chooses ReportMostActiveAuthors 4.1 The system calculates the most active authors 5. if the administrator chooses ReportTagPiuUsati 5.1 The system calculates the most used tags 6. if the administrator chooses ReportPoesiePiuInterazioni 6.1 The system calculates the poems with the most interactions 7. The system shows the administrator the selected report
Post-Conditions	The administrator views the report
Related use cases	InsertReportData
Sequence of alternative events	At point 3.1 if the administrator enters End Date<Start Date the system returns an error message

2.6.CLASS DIAGRAM

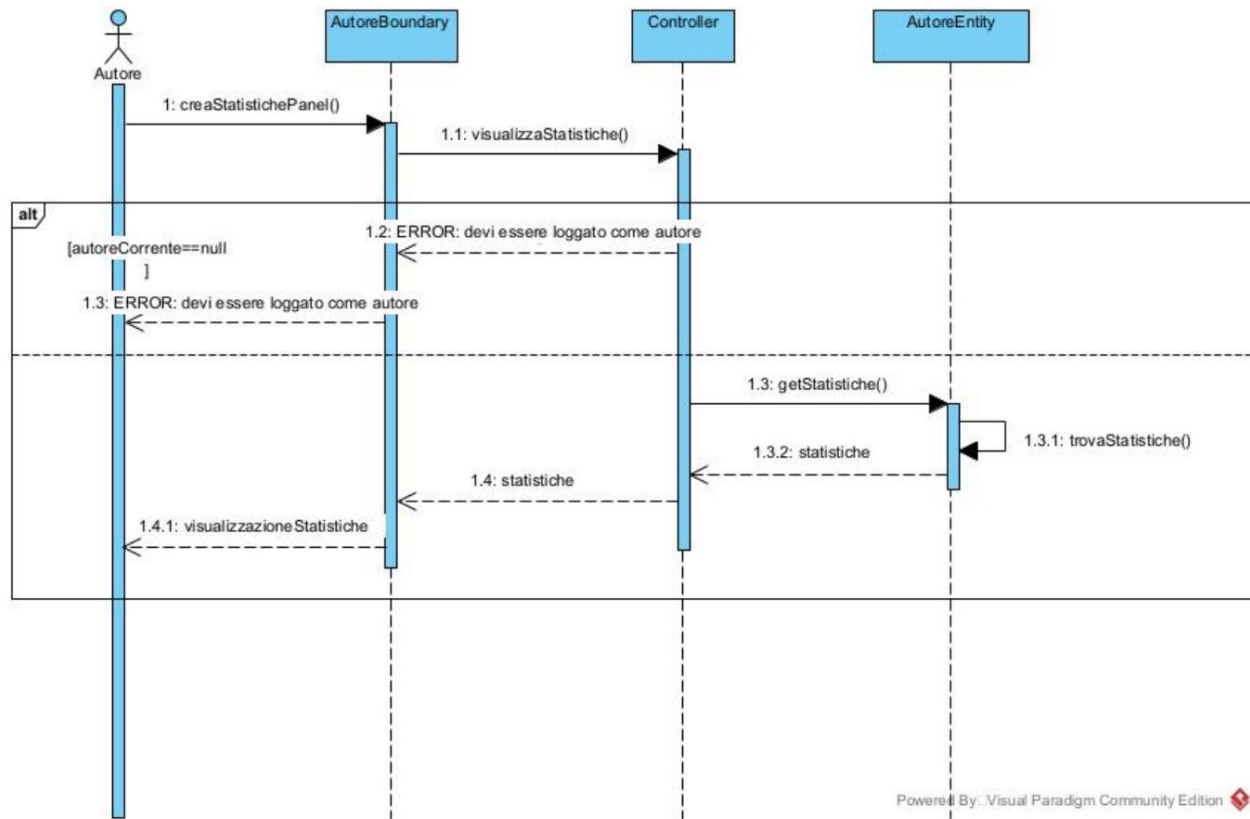


2.7. SEQUENCE DIAGRAMS

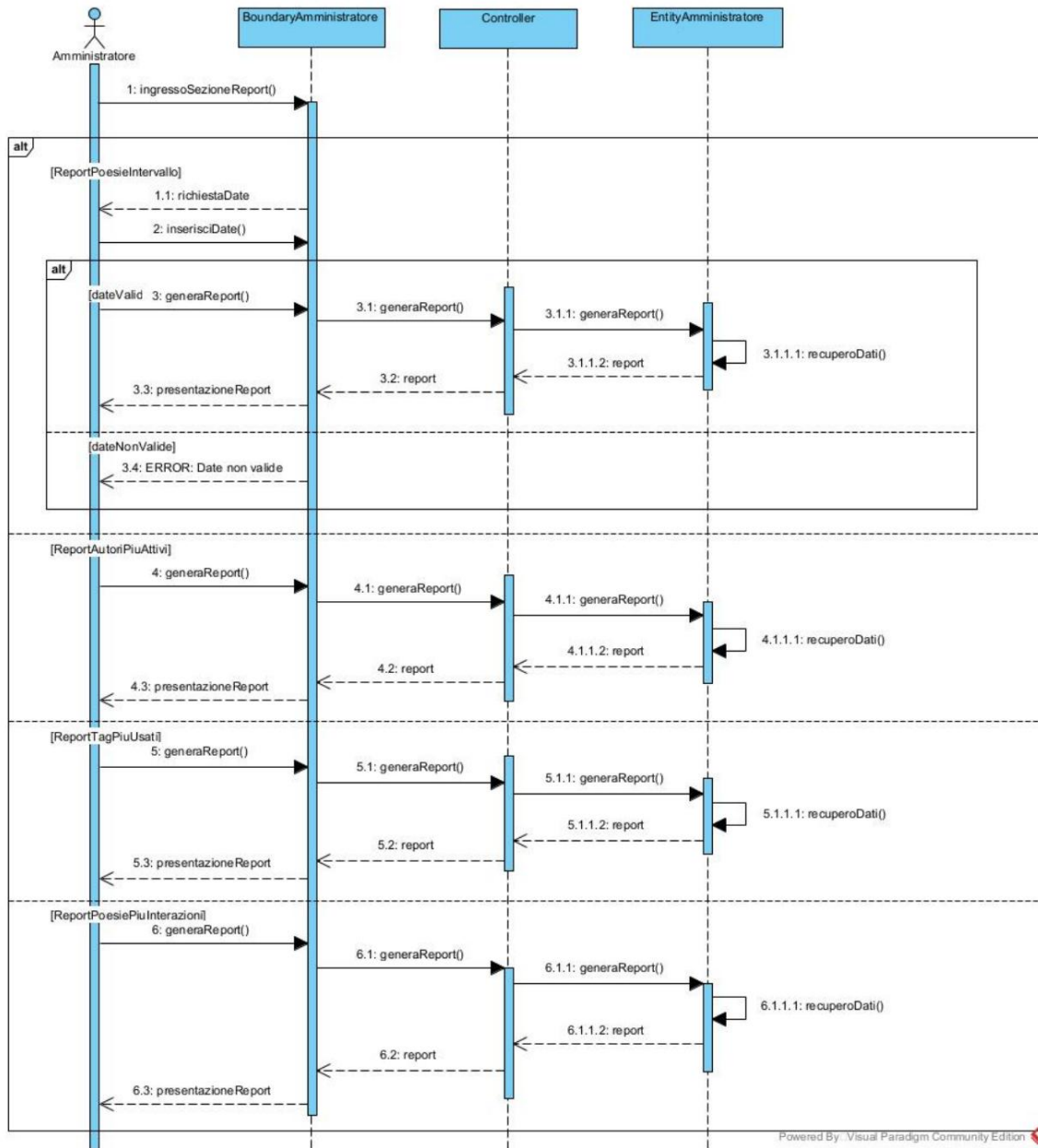
2.7.1. PUBLIC POETRY



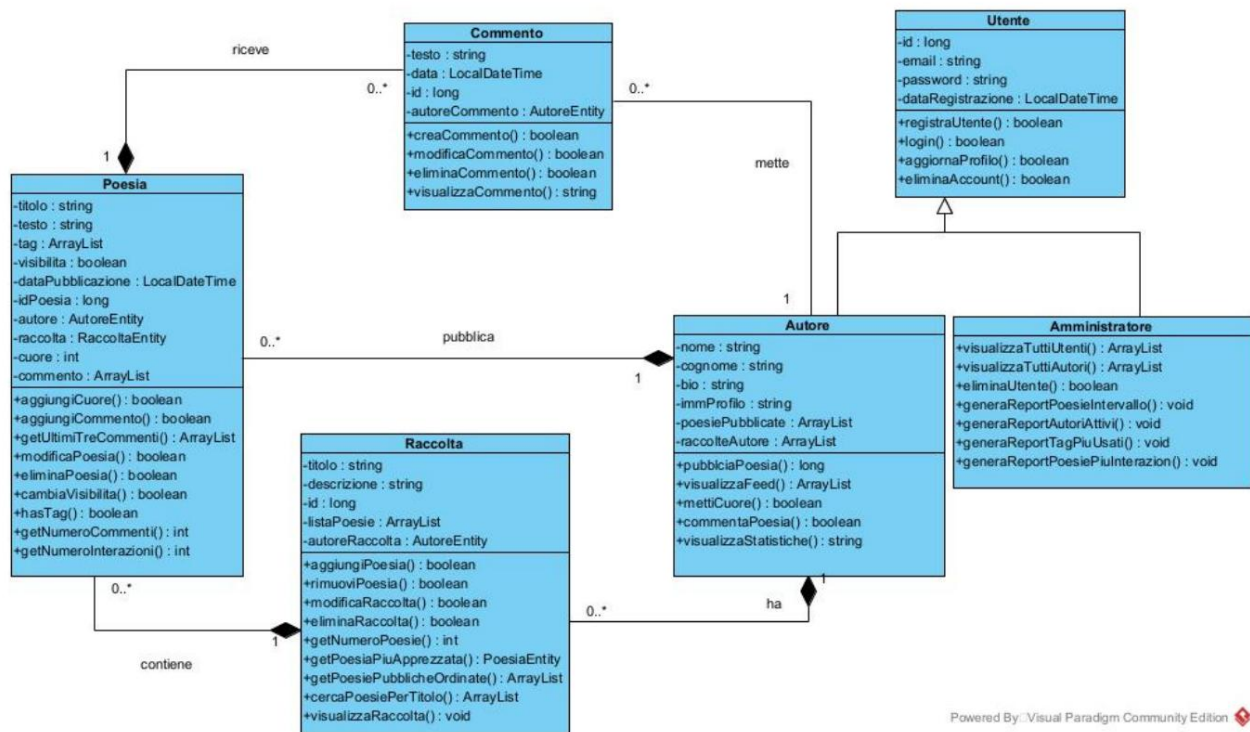
2.7.2. VIEW STATISTICS



2.7.3. GENERATE REPORTS



2.8. REFINED CLASS DIAGRAM



Powered By: Visual Paradigm Community Edition

3. Functional Test Plan

We intend to design functional test cases using the Category Partition Testing technique.

3.1.PUBLIC POETRY

Publish Poetry

Poetry	Title	Tags	SelectCollection
Character string of length <=500	Character string of length <=40	Character string of length <=40	The field must be enhanced
Character string of length >500 [ERROR]	Character string of length >40 [ERROR]	Character string of length >40 [ERROR]	The field is not valued [ERROR]
Character string of length ==00 [ERROR]	Character string of length ==00 [ERROR]	Character string with special characters [ERROR]	
		Character string of length ==00 [ERROR]	

The number of tests to be carried out without particular constraints is: $3*3*4*2=72$

With constraints instead the number of tests to be performed to test constraints is: 8

So the number of tests is: $(1*1*1*1)+8=9$

Test suite

T And st C to S And ID	Description ion	Equivalence classes covered fishing line And	Pre - with then Uncle no	Input	Out put atte Yes	Post t- with then Uncle no att ese
T C 1	Valid input	Valid poetry Valid title Valid tag Collect valid ta		{Poem: "I light up of immense", Title:"Morning", Tag:"Incommunicate rebirth, rebirth", Collection: "Introspection"	Poe both pub blic ata	There bit Well yes ae faces bile
T C 2	String poetry >500 characters you were	String poetry >500 characters [ERROR R] Title, Tag, Collect you valid		{Poem: "Miiii[...] ", Title: "Morning", Tag: "Incomu nicability, rebirth", Collection: "Introspection"	Poe it's too much po lung to!	
T C 3	String poetry ==0car wings	String poetry ==0car wings [ERROR R] Title, Tag, Collect you valid		{Poem: "",Title: "Morning",Tag: "Incommunicability a, rebirth", Collection: "Introspection"	Acts uncle ne, inse to laugh poe is!	
T C 4	Title string> 40 characters	Title string> 40 characters		{Poem: "I am illuminated by the immense", Title: "Morning aa aaaa", Tag: "Incommunicability,	Title or trop bit	

	you were	you were[ERR OR] Poetry, Tag, Collect you valid		rebirth", Collection: "Introspection"	long or!	
T C 5	Title string= =0 char you were	Title string= =0 char you were[ERR OR] Poetry, Tag, Collect you valid		Poetry: "I am enlightened by the immense", Title: "", Tags: "Incommunicability, rebirth", Collection: "Introspection"	Acts uncle ne, inse laugh title or!	
T C 6	Tag string>40 chars you were	Tag string>40 chars you were[ERR OR] Poetry, Title, Collect you valid		{Poem: "I am illuminated by the immense", Title: "Morning", Tag: "Incommunicate bilitaaaaaaaaaaaaaaaaaaaaaaaaaaaaa , reiiiiiiiibirth", Collection: "Introspection"	Tag trop po lung or!	
T C 7	Tag string with character you were special -	Tag string with character you were special i[ERROR R] Poetry, Title, Collect you valid		{Poem: "I am illuminated by the immense", Title: "Morning", Tag: "Incomunica rebirth, rebirth", Collection: "Introspection"	For mat or tag he's wrong to	
T C 8	Tag string==0 chars you were	Tag string==0 chars you were[ERR OR] Poetry,		{Poem: "I am illuminated by the immense", Title: "Morning", Tag: "", Collection: "I introspection"	Acts uncle ne, inse laugh tag!	

		Title, Collect				
		you valid				
T C 9	... camp or Management neRac cultured it is not valorized ato	Poetry, Title, Valid Tags,R welcomed not to valid [ERROR R]		{Poem: "I am illuminated by the immense", Title: "Morning", Tag: "Incommunicate rebirth, rebirth", Collection:""	Acts uncle ne, inse to laugh at a rac colt to	

3.2.VIEW STATISTICS

TEST PLAN USING THE CATEGORY-PARTITION TESTING METHOD FOR THE
FEATURES View Statistics

Authentication Number	of Poems Published	Number of Hearts Totals	Number Comments Totals	Poetry More Appreciated
Author Authenticated	Number Integer > 0	Number Whole>0	Number Whole>0	There is only one poem
Author not Authenticated [ERROR]	No Poetry Published (n=0)	No hearts received (n=0)	No comments received (n=0)	There are more poems (tie)

The number of tests to be carried out without particular constraints is: 2 * 2 * 2 * 2 * 2 = 32

We introduce the [ERROR] constraints and special cases (values of 0 or ties).

The number of tests to run to individually test the constraints is 5 (1 for Authentication, 1 for Number of Poems, 1 for Total Hearts, 1 for Total Comments, 1 for Uniqueness).

The resulting number of tests is: (1 * 1 * 1 * 1 * 1) + 5 = 6

TEST SUITE

Test Description	Classes of	Pre-	Input	Output	Post-
------------------	------------	------	-------	--------	-------

House and ID	And	Equivalena blankets	Conditions		expected	condition I waited
1	All valid inputs	Valid authentication Poems>0 Hearts>0 Comments>0 Uniqueness defined		{ Session: Active Poems[P1(Hearts:10 , Comment i:2)] }	Total Hearts: 10 Total Comments: 2 Top Poetry: P1	The user views the statistics and requests correct mind
TC2	Author Not Authentic to	Authentication not carried out Poems, Hearts, Comments, Unimportant uniqueness.		{Session: null}	Error message - Author not logged in.	The author is taken to the Login page.
TC3	Published Poems e = 0	Valid authentication Poems=0 Hearts=0 Comments=0 Undefined uniqueness		{Session: Active Poems [] }	Total Hearts: 0 Total Comments: 0 Top Poetry: "None Poetry"	The user views the statistics and nothing
TC4	Total Hearts=0	Valid authentication Poems>0 Hearts = 0 Comments>0 Undefined uniqueness		{ Session: Active Poems[P1(Hearts: 0, Comment i: 3)] }	Total Hearts: 0 Total comments: 3 Top Poetry: "Your poems have no Still received hearts"	The user views his own statistics and a message I inform myself I go to the section PoetryTo p
TC5	Total Comment i=0	Valid authentication Poems>0 Hearts>0 Comments=0 Uniqueness Defined		{ Session: Active Poems[P1 (Hearts : 4, Comment i:0) }]	Total Hearts:4 Total Comments: 0 PoetryTop: P1	The user views the statistics And correct requests

						mind
TC6 Draw	Max Hearts	Authentication and valid Poems>0 Hearts>0 Comments>0 Uniqueness: Tie		{ Session: Active Poems[P1(Hearts: 1, Comment i:0) P2(Hearts: 1 Comment i: 3)] }	Total Hearts:2 Total Comments: 4 PoetryTop: P2	The user views the statistics and requests(the system solves the draw or show (I'll give the most recent one)

3.3.GENERATING REPORTS

TEST PLAN USING THE CATEGORY-PARTITION TESTING METHOD FOR THE
GenerateReport FEATURES

AUTHENTICATION RANGE	DATES	CONTENT INTERVAL	CLASSIFICATION AUTHORS ACTIVE	MORE POEMS INTERACT	TAG MORE USED
Authenticated Administrator	End Date>=Date Start	Number of poems>0	5 different most active authors	Hearts or comments >0	Tags used >0
Administrator not authenticated [ERROR]	End Date<Date Start [ERROR]	No poetry in the interval or	Author with 0 poems (excl. use from the ranking)	No interaction with poetry	No tags used
			Authors with the same number of poems (TEST TIE)		

The number of tests to be performed without particular constraints is: $2*2*2*3*2*2=96$

We introduce the [ERROR] constraints and special cases (values of 0 or ties).

The number of tests to run to individually test the constraints is 7 (1 for Authentication, 1 for Date Range, 1 for Content Range, 2 for Active Author Ranking, 1 for Most Interacted Poems, 1 for Most Used Tags).

So the number of tests is: $(1*1*1*1*1*1)+7=8$

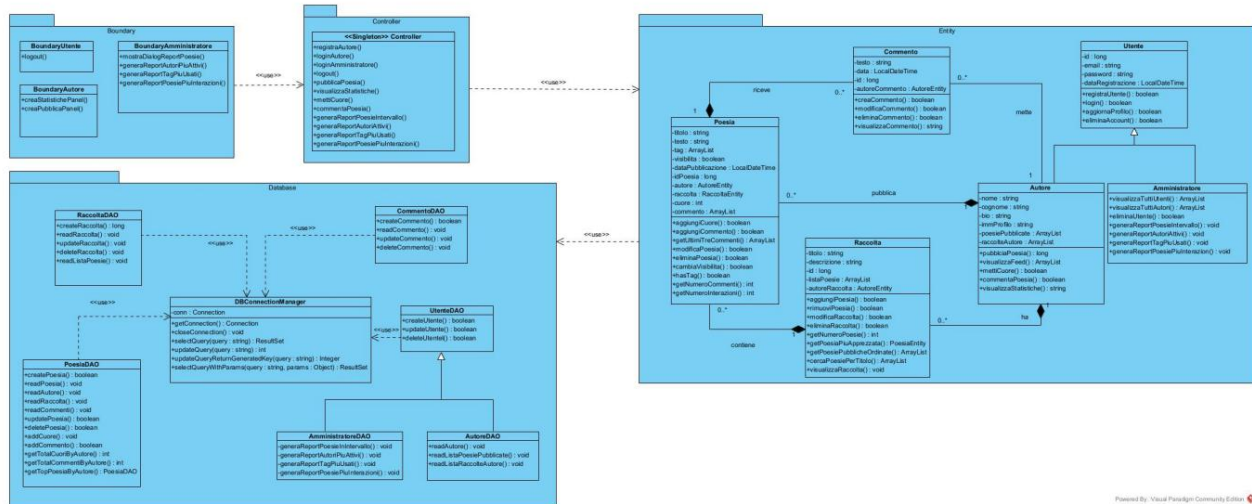
Test suite

Id Test	Description	Equivalence classes covered	Preconditions	Input	Expected outputs
TC1 All	inputs valid	Authenticated Administrator, end date>=start date, poems in range >0, 5 different active authors, interactions >0, tags used >0	Database populated with data	Auth-->ok Date: 2025-03-01 --> 2025-03-31	4 reports generated correctly nte
TC2 Error access		Administrator Not authenticated[ERROR R], end date>=start date, 5 different active authors, interactions>0, tags used>0	User not logged in correctly	Auth->no Error	Message: "Administrator actor not logged in correctly nte"
TC3 Validation Error and date		End Date<Start Date[ERROR], Authenticated admin, 5 different active authors, interactions > 0, tags used > 0	Administrator authenticated	Start date: 2025-03-01 Date End: February 2025 01	Error message: "The date range is invalid"
TC4 No poetry in the interval		Authenticated admin, end date>=start date, poems in range=0, 5 different active authors, interactions>0, tags used>0	Authenticated Administrator	Auth-->ok Date: 2025-03-01 --> 2025-03-31	The report with poems in interval has number of poems =0, the other reports are generated
TC5 Author with 0		Administrator authenticated, Administrator	date	Auth-->ok	The report with authors

	published poems	end>=start date, poems in range=0, author has no publications, interactions>0, tags used>0	authenticated, author Anna Neri with 0 published poems	Date: 2025-03-01 --> 2025-03-31	more active does not report 'Anna Neri', the other reports I am generated
TC6	Authors with the same number of published poems	Authenticated admin, end date>=start date, poems in range=0, authors same number of publications, interactions>0, tags used>0	Certified Administrator, Author Publications 'Anna Neri'==author publications 'Laura Bianchi'	Auth-->ok Date: 2025-03-01 --> 2025-03-31	The report with most active authors or reports 'Anna Neri' and 'Laura Bianchi' at the same position, the other reports are generated
TC7	No interaction	Authenticated admin, end date>=start date, poems in range =0, authors same number of publications, interactions for a poem =0, tags used>0	Authenticated administrator, poem 'x' has no interactions	Auth-->ok Date: 2025-03-01 --> 2025-03-31	Poem 'x' will not be present in the report More poems Interact, the other reports are generated
TC8	No Tag used	Authenticated admin, end date >= start date, poems in range =0, authors same number of publications, interactions for a poem =0, tags used for a poem =0	Authenticated Administrator, poem 'x' has no tags	Auth-->ok Date: 2025-03-01 --> 2025-03-31	Poem 'x' has no tags, the others report I am generated

4. Design

4.1.CLASS DIAGRAM



4.1.1. TRANSLATION OF CLASSES AND ASSOCIATIONS

The User class has been modeled as a superclass for Author and Administrator following the generalization/specialization mechanism, in particular the User class collects the attributes and methods common to all the actors of the system (such as email and password).

The Author class is a child class of the User superclass and therefore extends its specific functions such as those relating to the publication of poems, the management of collections and the display of statistics.

The Administrator class is a child class of the User superclass and therefore extends its specific functionality such as generating reports on aggregate data.

The Poetry class includes attributes such as title, text and tags, managed as ArrayList of String. Visibility was managed with a Boolean that in the interface is represented as a checkbox to be checked (True) if you want the poem you're about to publish to be public and visible to other users. This is then combined with the Author class so that if an author deletes themselves from the platform, their poems will also be deleted.

The Collection class has a compositional relationship with Poetry since deleting a collection will also delete all the poems within it.

The commentary was modeled as a class and not as a poem attribute since the commentary itself has its own attributes such as author and publication date.

Furthermore, this choice also helps to subsequently show the last 3 comments.

Furthermore, the Comment class is in composition with poetry because if you delete a poem, the related comments are also deleted.

Regarding associations:

Author 1 -- 0..* Poem indicates that an author can publish 0 or more poems but a certain poem can only be published by a certain author

Author 1 – 0..* collection indicates that an author can own 0 or more collections but a certain collection is only related to a certain author

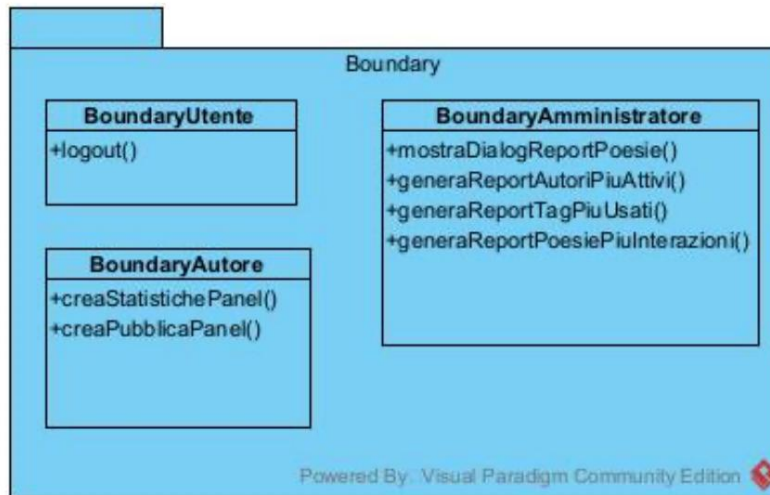
Author 1 – 0..* Comment indicates that an author can post 0 or more comments but a certain comment can only have been posted by a certain author

Collection 1 – 0..* Poem indicates that a collection can contain 0 or more poems but a certain poem can only be part of a certain collection

Poem 1 – 0..* Comment indicates that a poem can have 0 or more comments but a certain comment can only be related to a certain poem

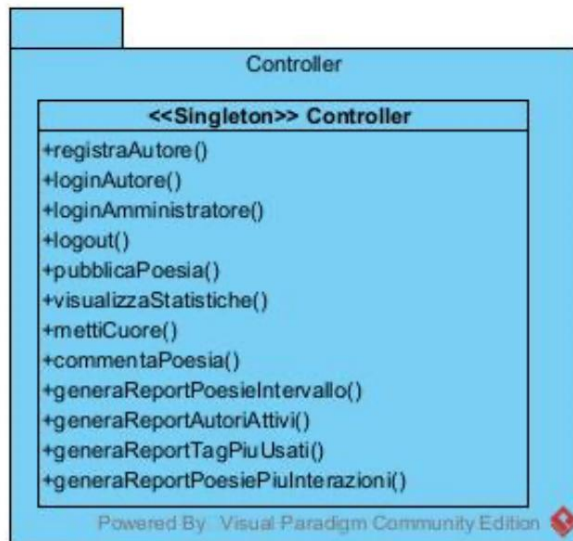
4.1.2. PATTERN BCED

4.1.2.1. PACKAGE BOUNDARY



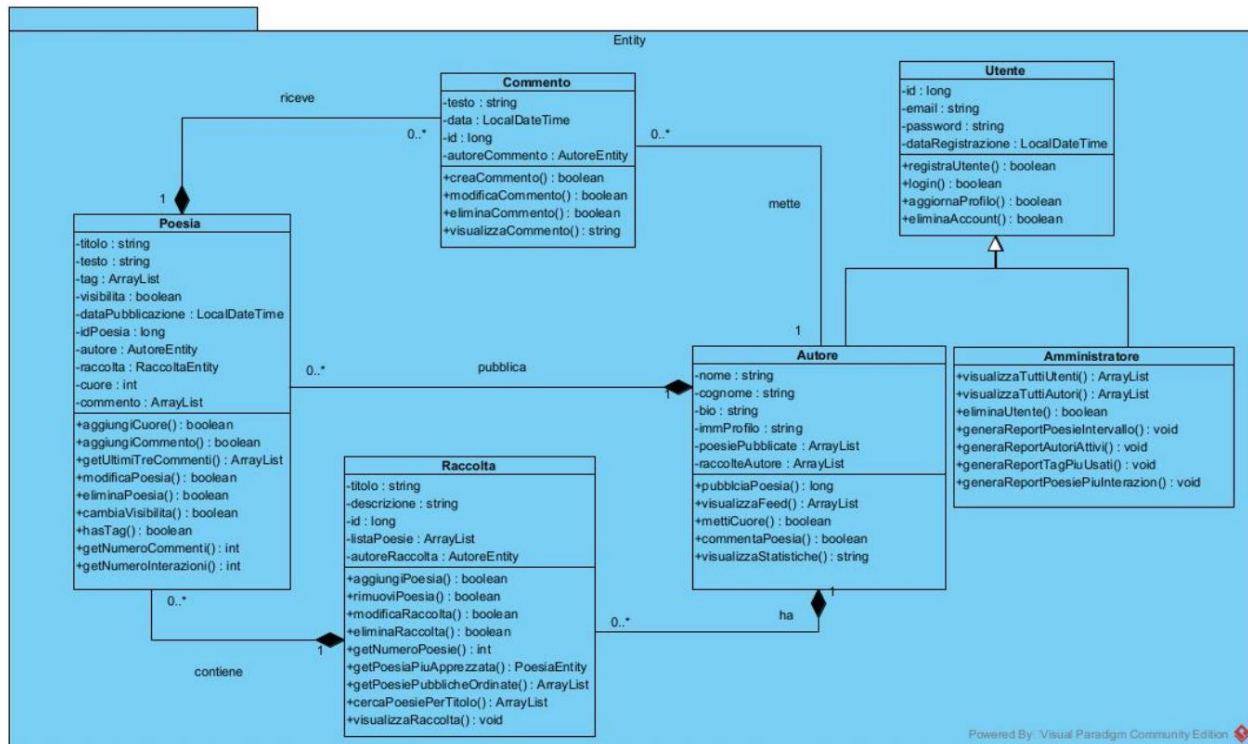
For this package, it was necessary to divide the interfaces based on the actor performing the access in order to be channeled towards the specific functional needs of the users and to limit access to unauthorized functions.

4.1.2.2. PACKAGE CONTROLLER



This is the "brain" that coordinates the flow of operations. This package contains a single singleton class called **Controller** to ensure a single point of access to the implemented logic, thus avoiding inconsistent states and simplifying dependency management.

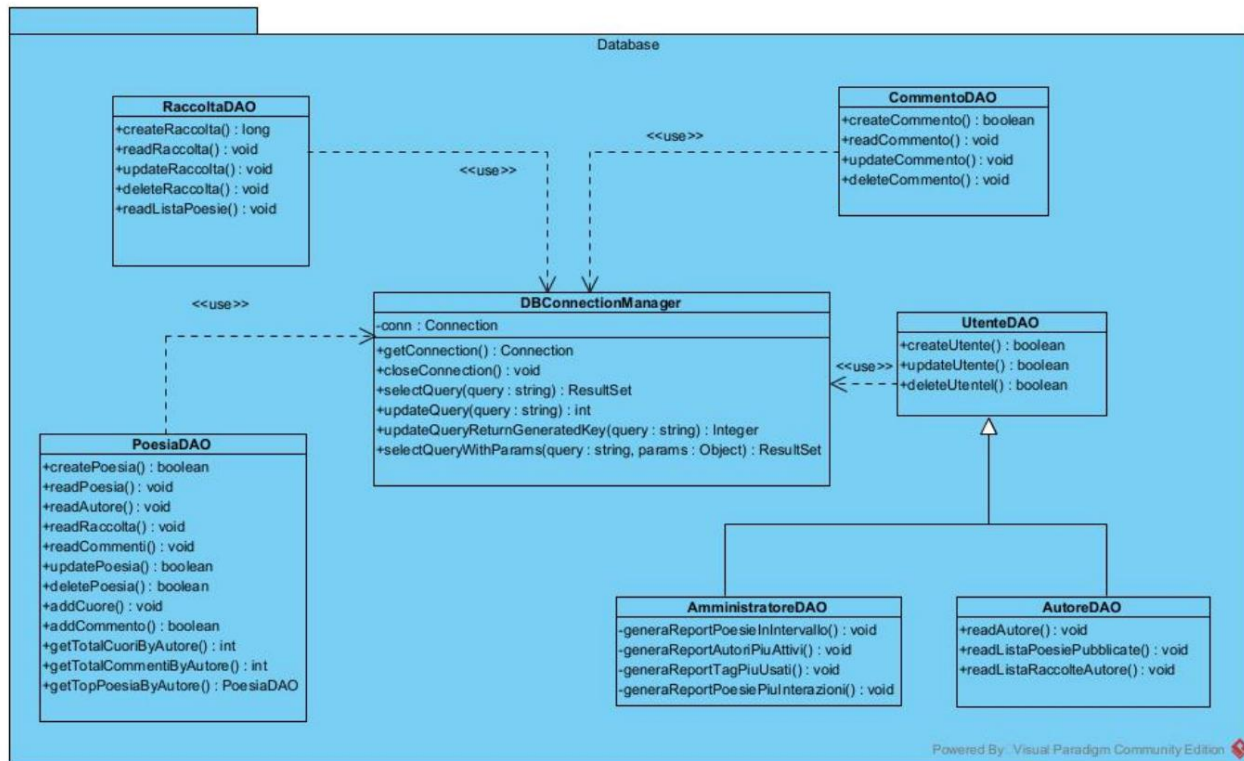
4.1.2.3. PACKAGE ENTITY



Entity classes represent the abstraction level in the BCED architecture. They embody the entities of the application domain, containing both the data and the associated business logic. Furthermore, relationships between the various entities in the domain are maintained at this level; for example, an instance of PoesiaEntity contains references to its AuthorEntity.

In this diagram we have shown the various classes with the related attributes and methods necessary for the correct functioning of the application, such as the validation methods that check the correctness of the data and business methods that implement the application-specific operations such as publishing, displaying and generating.

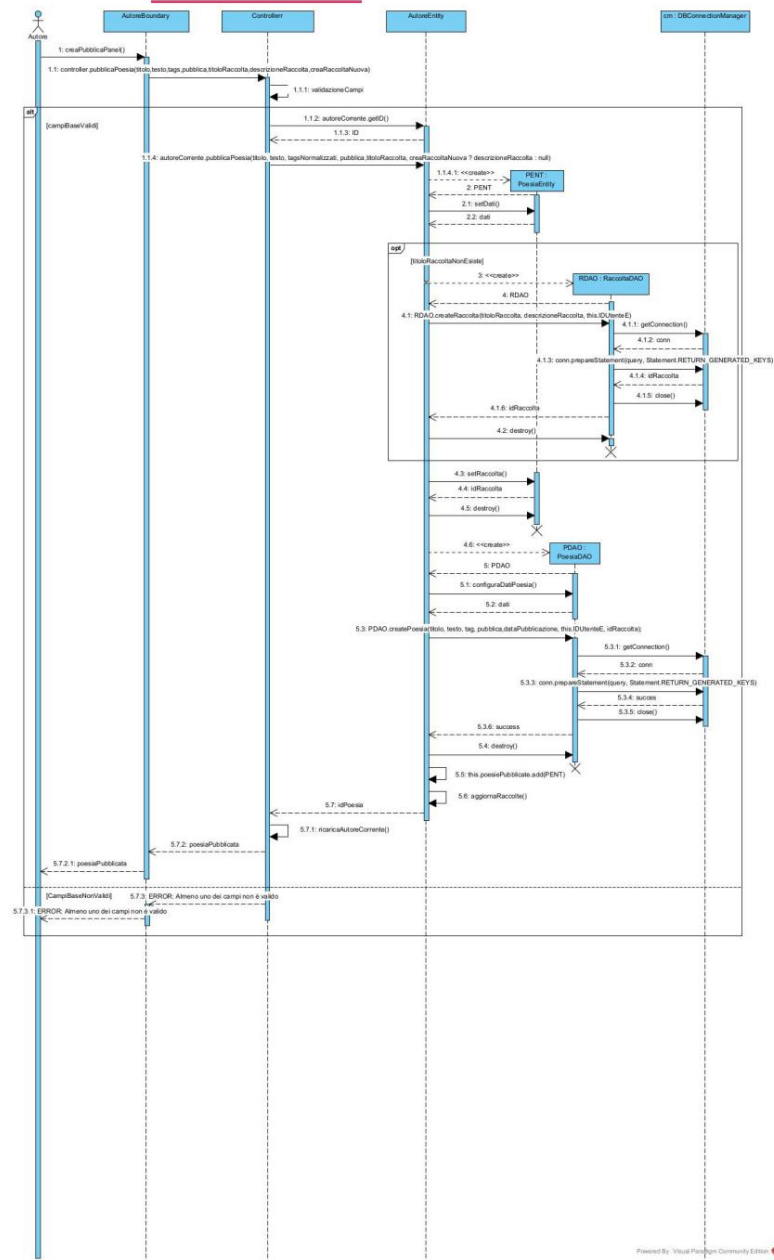
4.1.2.4. PACKAGE DATABASE



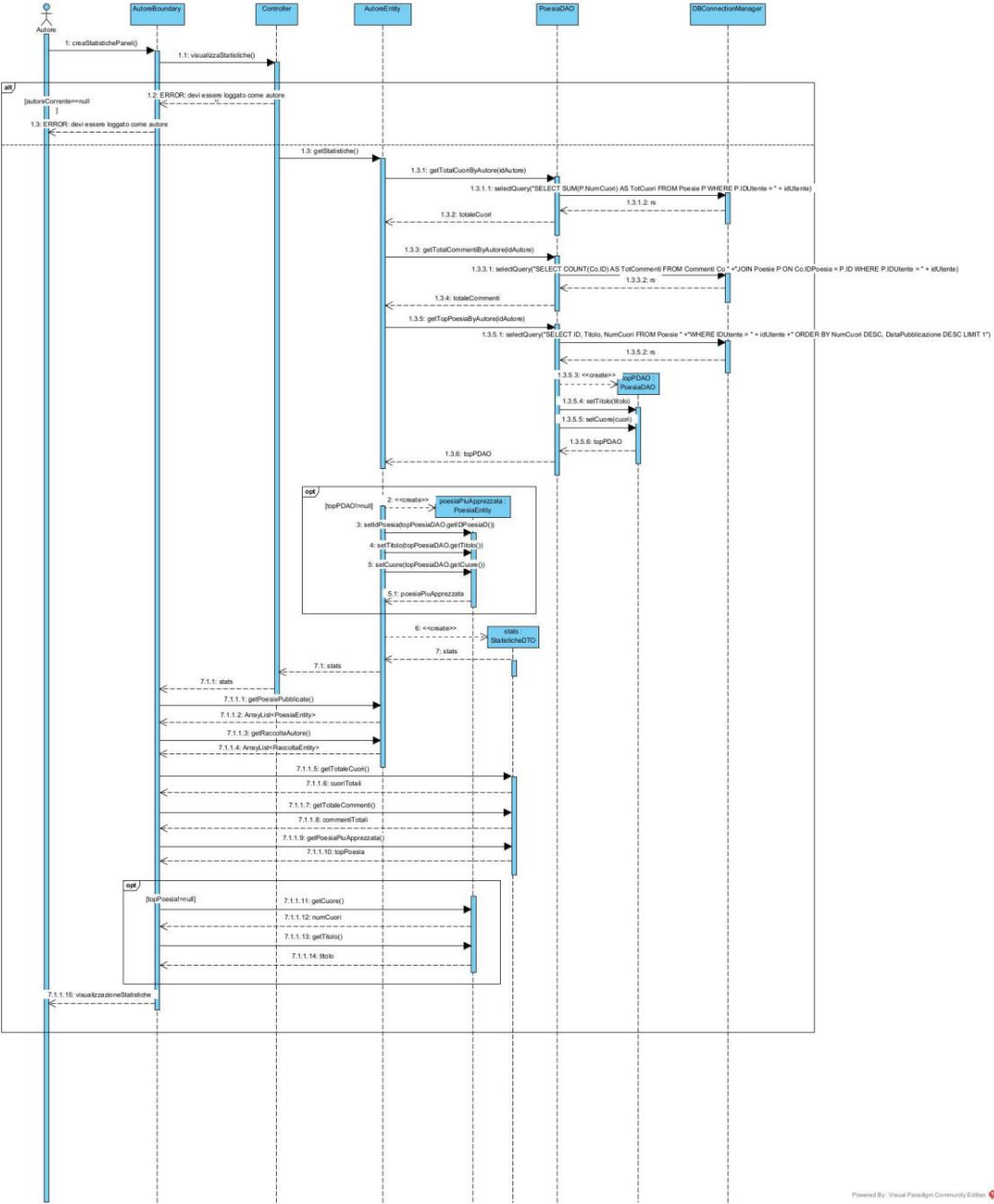
This package manages persistence, i.e., communication with the database, thanks to the **DBConnectionManager** class, which centralizes all connection and interaction operations with the MySQL database. This package implements the CRUD methods needed to translate the relational world of the database into the object-oriented world of Java, decoupling the logic from the database.

4.2. SEQUENCE DIAGRAMS

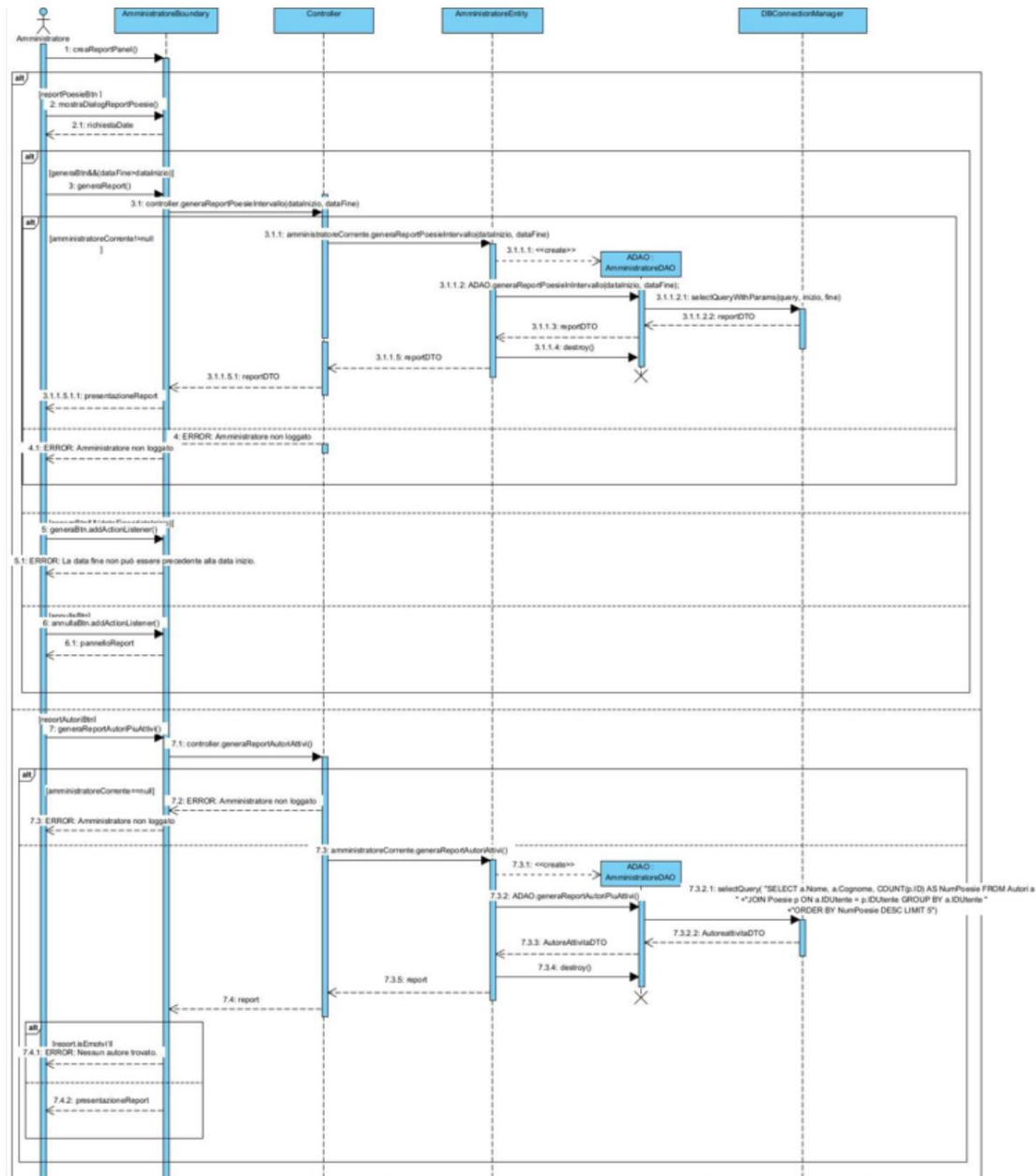
4.2.1. PUBLIC POETRY

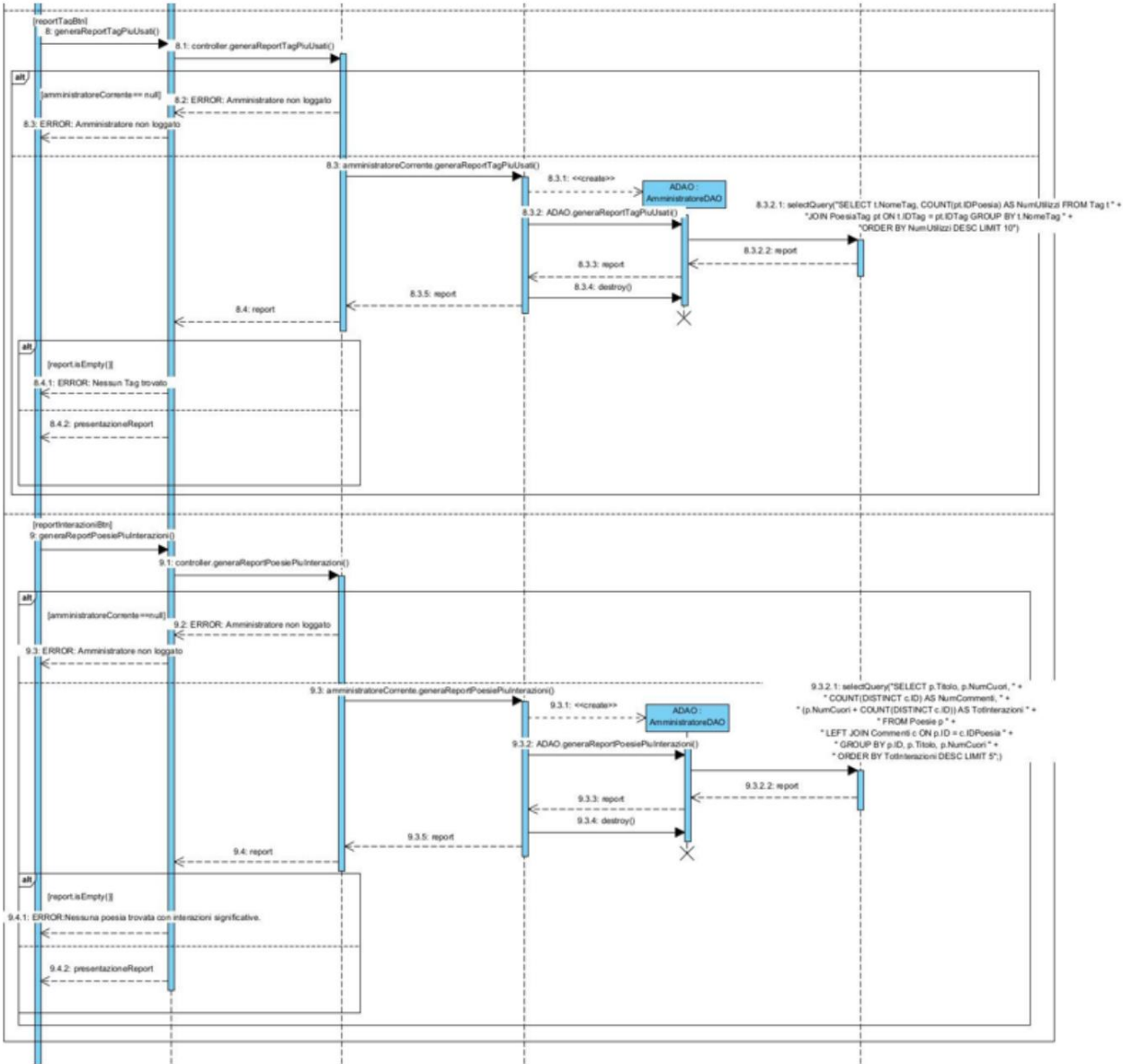


4.2.2. VIEW STATISTICS



4.2.3. GENERATE REPORT (2 SCREENS)





5. Implementation

Below is a description of the created packages and classes.

The software implementation was done using the Java programming language (version 20.0.1).

To enable communication with the MySQL relational DBMS, the mysql-connector-j 9.4.0 library was included in the project, distributed as a *JAR* file and used by the database management package. Database modeling, management, and testing were performed using the MySQL Workbench environment, which was used to create the schema, define tables, and test the queries used by the application.

The software implementation was carried out following the project diagrams, therefore based on a layered architecture according to the BCED pattern.

5.1. DATABASE PACKAGE

It provides direct interfacing with the database. Each persistent entity in the system is associated with a corresponding DAO class, which replicates its main attributes and manages data access operations. Relationships between entities are similarly reflected by appropriate associations between the respective DAOs.

DBMS connection management is centralized in the static *DBConnectionManager* class, which is responsible for creating and maintaining the connection to the MySQL system. This class provides methods for:

- the correct closure of the connection;
- building securely formatted queries, preventing attempts intentional or accidental SQL Injection;
- sending queries to the DBMS to perform selection operations, update, remove and insert, as well as to recover any keys automatically generated following the execution of SQL statements.

Within this package the following classes have been defined:

- Ø *DBConnectionManager*: responsible for connecting to the database
- Ø *DAO User*: responsible for extracting, reading and updating data relating to a Database User
- Ø *AuthorDAO*: responsible for extracting, reading and updating data related to a Database Author
- Ø *DAO Administrator*: Responsible for extracting, reading and updating data related to a Database Platform Administrator

- ✗ PoesiaDAO: responsible for extracting, reading and updating data related to a Database Poem
- ✗ CollectionDAO: responsible for extracting, reading and updating data related to a Database Collection
- ✗ CommentoDAO: responsible for extracting, reading and updating data related to a comment of a poem in the Database

5.2.PACKAGE ENTITY

It implements the classes previously defined in the class diagram. In particular, it contains the classes that represent the system domain objects, with related functions, and act as intermediaries with the database package.

Within this package the following classes have been defined:

- ✗ EntityUser: responsible for the management and storage of data relating to a User
- ✗ EntityAuthor: responsible for the management and storage of data relating to an Author
- ✗ Entity Administrator: responsible for data management and storage relating to an Administrator
- ✗ EntityPoesia: responsible for the management and storage of data relating to a poem
- ✗ EntityRaccolta: responsible for the management and storage of data relating to a Collection
- ✗ EntityCommento: responsible for managing and storing data relating to a Comment

5.3.PACKAGE CONTROL

It implements the system's responsibilities (identified in the use case analysis phase) and contains all the methods that implement the previously highlighted use cases, in a Controller class that adopts the Singleton pattern.

The class presents as attributes every object or collection of objects that is needed

maintain interaction with the GUI. The following class has been defined within this package:

Controller : controls the execution of a process. The pattern has been chosen. singleton so that you define and use a single instance that will be created only once and called when it needs to be used.

5.4. PACKAGE BOUNDARY

It handles user interfacing, meaning it contains classes that represent the interfaces for each user. The administrator will interface with BoundaryAdministrator to access their section and dedicated services. An author, on the other hand, will interact with BoundaryAuthor to use the services dedicated to existing users.

Register on the platform. Finally, an unregistered user will interact with BoundaryUser, which is the same GUI used to log in to the platform for both the administrator and the author. The following classes have been defined within this package:

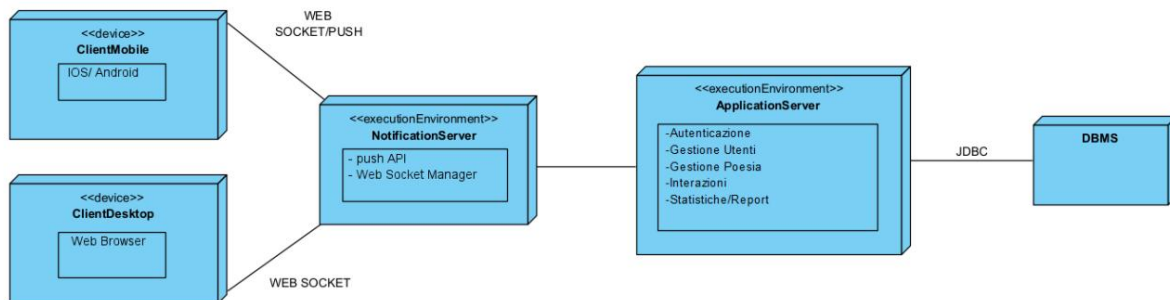
- Ø MainFrame: represents the main graphical interface
- Ø UserBoundary: responsible for the functionality presentation logic available to a User
- Ø BoundaryAdministrator: responsible for the presentation logic of the available features of an Administrator
- Ø BoundaryAuthor: responsible for the functionality presentation logic available from an Author
- Ø NotificationManager: responsible for the logic of sending/receiving notifications

5.5. DTO PACKAGE

It's a design pattern used to transfer data between different layers of an application (frontend->backend). It optimizes communication between client and server, reduces overhead (avoids passing overly large objects), and increases security (the client is given only what it needs). Entity classes specifically contain internal fields, relationships, or constraints that shouldn't always be accessible outside the backend, so DTO classes are created, including:

- Ø AutoreAttivitaDTO: responsible for the management and storage of the data used in the generation of the ReportAutoriPiuAttivi
- Ø ReportIntervalloDTO: responsible for the management and storage of the data used in the generation of the ReportPoesieIntervallo
- Ø StatisticsDTO: responsible for the management and storage of the related data to the Statistics of a given Author

5.6. DEPLOYMENT DIAGRAM



6. Testing

6.1. STRUCTURAL TEST

6.1.1. CYCLOMATIC COMPLEXITY

The cyclomatic complexity of the Controller publicPoesia(...) method is then evaluated.

```

public long pubblicaPoesia(String titolo, String testo, ArrayList<String> tags,
                          boolean pubblica, String titoloRaccolta,
                          String descrizioneRaccolta, boolean creaRaccoltaNuova) {
    if (autoreCorrente == null) {
        JOptionPane.showMessageDialog(null, "Devi essere loggato come autore per
        pubblicare una poesia!", "ERRORE", JOptionPane.ERROR_MESSAGE);
        return -1;
    }

    try {
        validazioneStringa(titolo, Contesto.TITOLOP);
        validazioneStringa(testo, Contesto.TESTOP);
        if (tags == null || tags.isEmpty()) {
            throw new IllegalArgumentException("Devi inserire almeno un tag!");
        }

        for (String tag : tags) {
            validazioneStringa(tag, Contesto.TAG);
        }

        if (titoloRaccolta == null || titoloRaccolta.trim().isEmpty()) {
            throw new IllegalArgumentException("Devi specificare una raccolta!");
        }
        validazioneStringa(titoloRaccolta, Contesto.TITOLOR);

        if (creaRaccoltaNuova) {
            if (descrizioneRaccolta == null || descrizioneRaccolta.trim().isEmpty()) {
                throw new IllegalArgumentException("Devi specificare la descrizione
                per la nuova raccolta!");
            }
            validazioneStringa(descrizioneRaccolta, Contesto.DESCRIZIONER);

            if (raccoltaEsiste(titoloRaccolta)) {
                throw new IllegalArgumentException("Esiste già una raccolta con questo titolo!
                Scegli un altro nome o seleziona la raccolta esistente.");
            }
        } else {
            if (!raccoltaEsiste(titoloRaccolta)) {
                throw new IllegalArgumentException("La raccolta '" + titoloRaccolta + "' non esiste!
                Crea prima o scegli un'altra raccolta.");
            }
        }

        ArrayList<String> tagsNormalizzati = new ArrayList<>();
        for (String tag : tags) {
            tagsNormalizzati.add(tag.trim().toLowerCase());
        }

        long idAutoreCorrente = autoreCorrente.getID();

        try {
            long idPoesia = autoreCorrente.pubblicaPoesia(titolo, testo, tagsNormalizzati, pubblica,
            titoloRaccolta, creaRaccoltaNuova ? descrizioneRaccolta : null);

            if (idPoesia != -1) {
                this.autoreCorrente = new AutoreEntity(idAutoreCorrente);
                this.utenteCorrente = this.autoreCorrente;
            }

            return idPoesia;
        } catch (Exception e) {
            JOptionPane.showMessageDialog(null,
            "Errore durante la pubblicazione: " + e.getMessage(),
            "ERRORE DATABASE", JOptionPane.ERROR_MESSAGE);
            return -1;
        } catch (IllegalArgumentException | UnknownHostException e) {
            JOptionPane.showMessageDialog(null, e.getMessage(), "ERRORE", JOptionPane.ERROR_MESSAGE);
            return -1;
        }
    }
}

```

(1) if
(2)
(3) exit
(4) try
(5) PossibileEccezione
(6) PossibileEccezione
(7) if
(8) Eccezione
(9) for
(10) PossibileEccezione
(11) if
(12) Eccezione
(13) PossibileEccezione
(14) if
(15) if
(16) Eccezione
(17) PossibileEccezione
(18) if
(19) Eccezione
(20) if
(21) Eccezione
(22)
(23) for
(24)
(25)
(26) try
(27) PossibileEccezione
(28) if
(29) PossibileEccezione
(30)
(31) exit
(32) Catch
(33)
(34) exit
(35) Catch
(36)
(37) exit

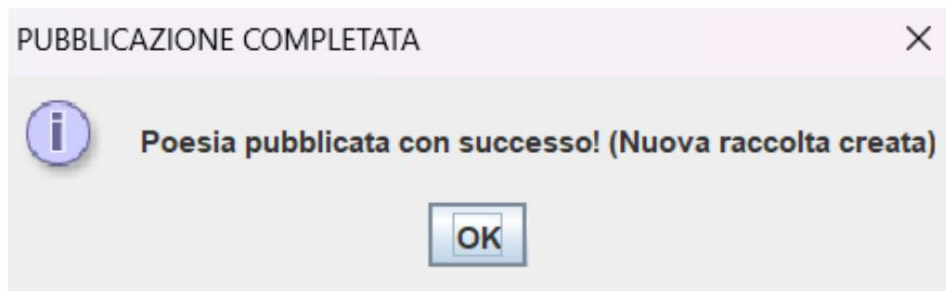
§ number of closed regions of the graph + 1 = 18+1=19
 § number of predicate nodes +1 = 18+1=19 §
 # edges – # nodes + 2 = (54- 37) + 2 = 17+2=19

- 1) 1-2-3-exit
- 2) 1, 4, 5, 35, 36, 37 -exit
- 3) 1, 4, 5, 6, 35, 36, 37 -exit
- 4) 1, 4, 5, 6, 7, 8, 35, 36, 37 -exit
- 5) 1, 4, 5, 6, 7, 9, 10, 35, 36, 37 -exit
- 6) 1, 4, 5, 6, 7, 9, 10, 9, 11, 12, 35, 36, 37 -exit
- 7) 1, 4, 5, 6, 7, 9, 10, 9, 11, 13, 35, 36, 37 -exit
- 8) 1, 4, 5, 6, 7, 9, 10, 9, 11, 13, 14, 15, 16, 35, 36, 37 -exit
- 9) 1, 4, 5, 6, 7, 9, 10, 9, 11, 13, 14, 15, 17, 35, 36, 37 -exit
- 10) 1, 4, 5, 6, 7, 9, 10, 9, 11, 13, 14, 15, 17, 18, 19, 35, 36, 37 -exit
- 11) 1, 4, 5, 6, 7, 9, 10, 9, 11, 13, 14, 20, 21, 35, 36, 37 -exit
- 12) 1, 4, 5, 6, 7, 9, 10, 9, 11, 13, 14, 20, 22, 23, 24, 23, 25, 26, 27, 32, 33, 34 -exit
- 13) 1, 4, 5, 6, 7, 9, 10, 9, 11, 13, 14, 20, 22, 23, 24, 23, 25, 26, 27, 28, 29, 32, 33, 34 -exit
- 14) 1, 4, 5, 6, 7, 9, 10, 9, 11, 13, 14, 20, 22, 23, 24, 23, 25, 26, 27, 28, 29, 30, 31 -exit
- 15) 1, 4, 5, 6, 7, 9, 10, 9, 11, 13, 14, 20, 22, 23, 24, 23, 25, 26, 27, 28, 31 -exit

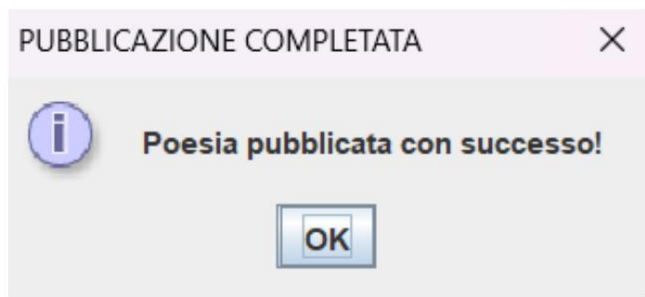
- 16) 1, 4, 5, 6, 7, 9, 10, 9, 11, 13, 14, 15, 17, 18, 22, 23, 24, 23, 25, 26, 27, 32, 33, 34 -exit 17) 1, 4, 5, 6, 7, 9, 10, 9, 11, 13, 14, 15, 17, 18, 22, 23, 24, 23, 25, 26, 27, 28, 29, 32, 33, 34 -exit
- 18) 1, 4, 5, 6, 7, 9, 10, 9, 11, 13, 14, 15, 17, 18, 22, 23, 24, 23, 25, 26, 27, 28, 29, 30, 31 -exit
- 19) 1, 4, 5, 6, 7, 9, 10, 9, 11, 13, 14, 15, 17, 18, 22, 23, 24, 23, 25, 26, 27, 28, 31 -exit

6.2.JUNIT

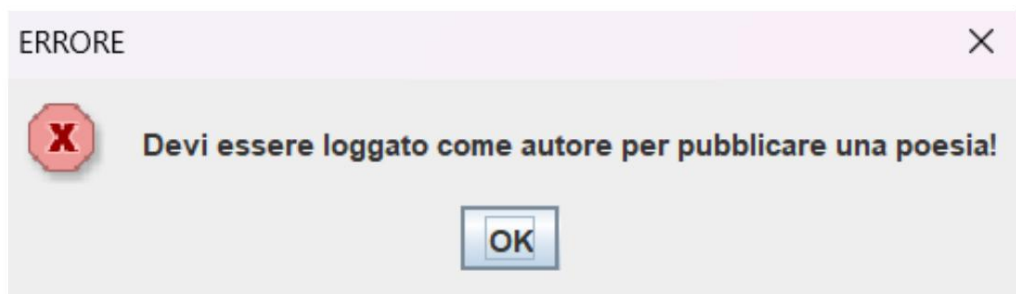
TC1



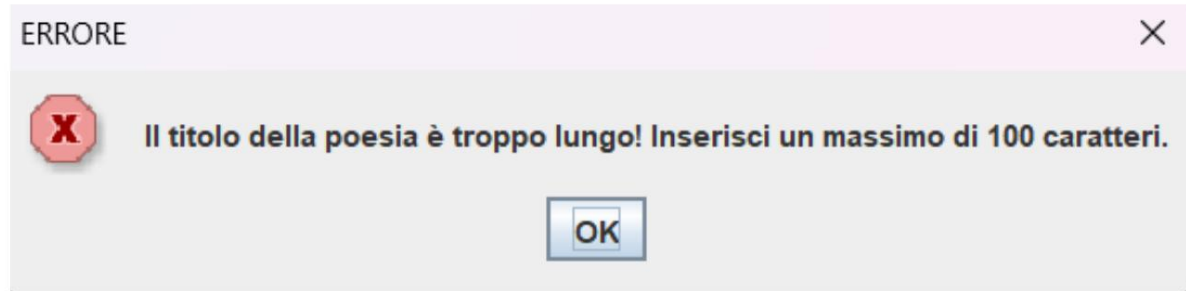
TC2



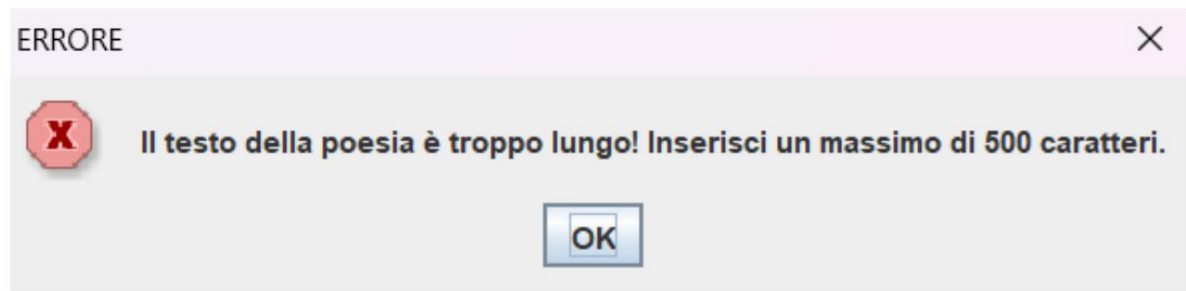
TC3



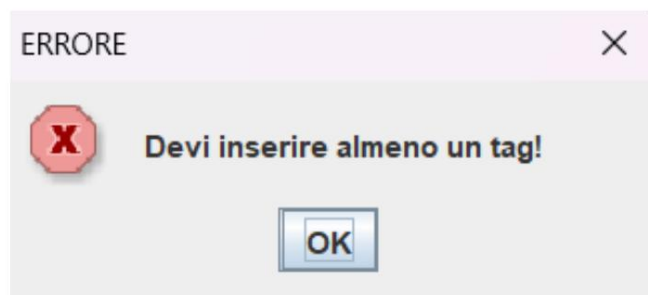
TC4



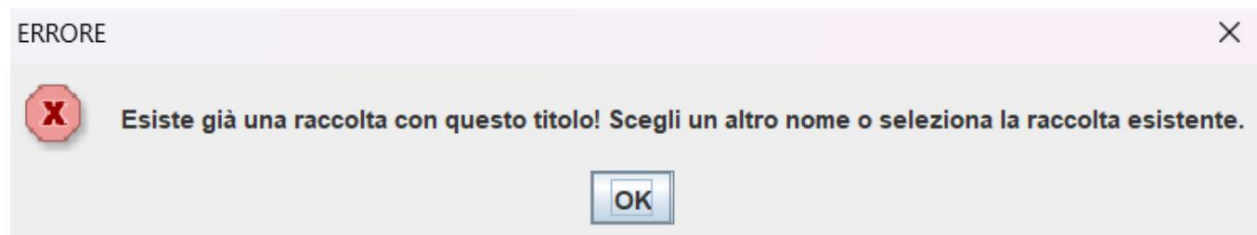
TC5



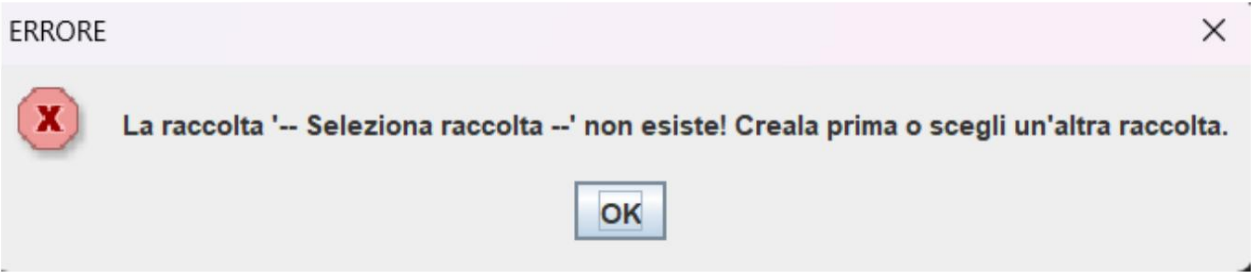
TC6



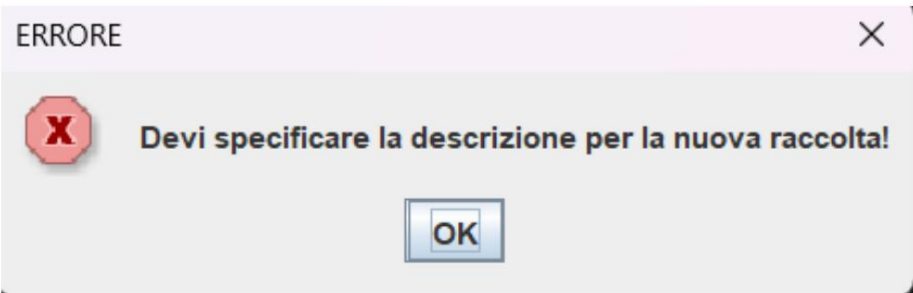
TC7



TC8



TC9



6.3. FUNCTIONAL TEST

tion	Equivalence classes covered	Pre- Conditions	Input	Expected outputs	Post- Expected conditions	Outputs obtained Post-	Condition obtained
lication ion at high ESS]	Author logged in Valid title Valid text Valid Tags CreateCollection Valid collection title Valid collection description		{Author: logged in, Title: "Love", Text: "I am illuminated with the immense", Tags: "Sun, Moon", Create Collection: True, Title Collection: "Hello", Collection description: "bye bye"	Published poetry and collection new created	The poem and the collection were saved in the system	Published poetry and new collection created	The poem and the collection were saved in the system
accepted application ESS]	Author logged in Valid title Valid text Valid Tags		{Author: logged in, Title: "Love", Text: "I am illuminated with the immense", Tags: "Sun, Moon",	Poetry published and included in the collection	The poem has been saved in the system	Poetry published and included in the existing collection	The poem has been saved in the system

	CreateCollection valid		Create Collection: False}	existing			
lication to with and not to	Author not logged in		{Author: null}	Author Not logged in to publish poetry		"You must be logged in as the author..."	
lication to with Not	Author logged in Title not valid Valid text Valid Tags		{Author logged in, Title: "Loveeee Eeeeeeeee eeeeEeee eeeeeee Eeeeeeeee eeee eeeeeeeeee ee, Text: "I am illuminated by the immense", Tags: "Sun, Moon", }	Unpublished poem cause invalid title		"The title of the poem is too long..."	
lication to with Not	Author logged in Valid title Text not valid Valid Tags		{Author logged in, Title: "Love , Text: "I light up of immense..", Tags: "Sun, Moon"}	Unpublished poem cause invalid text		"The text of the poem is too long..."	
lication to with on	Author logged in Valid title Valid text Invalid tags		{Author logged in, Title: "Love , Text: "I light up of immense..", Tags: null}	Unpublished poem due to tags invalid		"You must enter at least one tag"	
lication ion to Ita] Title once already nte	Author logged in Valid title Valid text Valid Tags CreateCollection valid Invalid collection title Valid collection description		{Author: logged in, Title: "Love", Text: "I am illuminated with the immense", Tags: "Sun, Moon", Create Collection: True, Title Collection: "Hello", Collection description: "bye bye"	The collection with that title already exists		"There's already a collection with this title!..."	

elective lication to lta] Title olta not	Author logged in Valid title Valid text Valid Tags Create valid Collection Invalid collection title Valid collection description		{Author: logged in, Title: "Love", Text: "I am illuminated with the immense", Tags: "Sun, Moon", Create Collection: True, Title Collection: "SpringPoems", Collection description: "bye bye"	The title of that collection does not exist		" The PoesiePrimaverili " collection doesn't exist! ...	
lication with high iction not to R]	Author logged in Valid title Valid text Valid Tags Create valid Collection Valid collection title Description of collection not valid		{Author: logged in, Title: "Love", Text: "I am illuminated with the immense", Tags: "Sun, Moon", Create Collection: True, Title Collection: "Hello", Collection Description: null}	You must enter a description for the collection.		"You must specify the description for the new collection!"	

